

B28: SQL SuperHero Program : Session 14

1 message

Vishal Jaiswal <vishaldbdev@gmail.com>
Bcc: satishhattekar1997@gmail.com

6 March 2025 at 00:12

ROW_NUMBER() --> this is used to generate a sequence number of records using analytical function. The ROW_NUMBER analytic function is order-sensitive and produces an error if you attempt to use it without an ORDER BY in the analytic clause. Row_number means : every row is a number. it will give a number to each row.

syntax of using analytical function:

Analytical_function () over (partition by <column name> order by <column name>)

() --> windowing clause

--> partition by will just like group by in normal sql

select B.*, row_number() over(order by brick_id) from bricks B

Arrange the data as per brick-id

BRICK_ID	COLOUR	SHAPE	WEIGHT	ROW_NUMBER()OVER(ORDERBYBRICK_ID)
1	blue	cube	1	1
2	blue	pyramid	2	2
3	red	cube	1	3
4	red	cube	2	4
5	red	pyramid	3	5
6	green	pyramid	1	6

6 rows selected.

After that row-number generates number

```
select B.*, row_number() over(order by brick_id desc) from bricks B
```

First
arrange
in
desc
order

BRICK_ID	COLOUR	SHAPE	WEIGHT	ROW_NUMBER()OVER(ORDERBYBRICK_IDDESC)
6	green	pyramid	1	1
5	red	pyramid	3	2
4	red	cube	2	3
3	red	cube	1	4
2	blue	pyramid	2	5
1	blue	cube	1	6

After that
row number
assigned

6 rows selected.

Now see this with partition By. It is written Partition By colour, it means when colour changes start again. what starts again??? row numbering starts again. first 2 records are BLUE colour then it assigned number 1,2 then colour changes to green then row number start again and assigns 1 and then colour changes again, now it is RED colour, so it was starts again and 3 rows so assigns row number as 1,2,3

→ color changes start again

```
select B.*, row_number() over(partition by colour order by brick_id desc) rownumber from bricks B
```

③

①

②

BRICK_ID	COLOUR	SHAPE	WEIGHT	ROWNUMBER
2	blue	pyramid	2	1
1	blue	cube	1	2
6	green	pyramid	1	1
5	red	pyramid	3	1
4	red	cube	2	2
3	red	cube	1	3

6 rows selected.

→ ① first create boundary
② then order
③ Then assign number

See the output of below query:

```
select H.*, row_number() over(order by salary) from hr.employees H where H.department_id in (90,60,100);
```

There is no partition by, means no boundary, so it assigns numbering 1,2,3..... to all records

`select H.*, row_number() over(order by salary) from hr.employees H where department_id in (60,90,100)`

EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID	ROW_NUMBER()OVER(ORDERBYSALARY)
DLORENTZ	590.423.5567	07-FEB-07	IT_PROG	4200	-	103	60	1
DAUSTIN	590.423.4569	25-JUN-05	IT_PROG	4800	-	103	60	2
VPATABAL	590.423.4560	05-FEB-06	IT_PROG	4800	-	103	60	3
BERNST	590.423.4568	21-MAY-07	IT_PROG	6000	-	103	60	4
LPOPP	515.124.4567	07-DEC-07	FI_ACCOUNT	6900	-	108	100	5
ISCIARRA	515.124.4369	30-SEP-05	FI_ACCOUNT	7700	-	108	100	6
JMURMAN	515.124.4469	07-MAR-06	FI_ACCOUNT	7800	-	108	100	7
JCHEN	515.124.4269	28-SEP-05	FI_ACCOUNT	8200	-	108	100	8

Now lets do this small change and introduce Partition By :

`select H.*, row_number() over(partition by department_id order by salary desc) from hr.employees H where H.department_id in (90,60,100);`

→ creates boundary on Partition

G	H	I	J	K	L	M	N
JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID	ROWNUMBER		
IT_PROG	9000	-	102	60	1	✓	
IT_PROG	6000	-	103	60	2	✓	
IT_PROG	4800	-	103	60	3	✓	1
IT_PROG	4800	-	103	60	4	✓	
IT_PROG	4200	-	103	60	5	✓	
AD_PRES	24000	-	-	90	1	✓	
AD_VP	17000	-	100	90	2	✓	2
AD_VP	17000	-	100	90	3	✓	
FI_MGR	12008	-	101	100	1		
FI_ACCOUNT	9000	-	108	100	2		
FI_ACCOUNT	8200	-	108	100	3		3
FI_ACCOUNT	7800	-	108	100	4		
FI_ACCOUNT	7700	-	108	100	5		
FI_ACCOUNT	6900	-	108	100	6		

Boundary Change

Then row number assigned

start again

Arrange data order by sal desc in that boundary

Suppose, we have to find out, who is the 2nd person joined the department.

As we discussed in class, row_number() is an analytical function, use to assign number for each row. consider, if i want to find out, who is the 2nd person, joined the department.

→ Whenever the ask is to find other than max and min then Analytical, 2nd, 3rd, 4th, etc.... analytical function is the answer

`select H.*, row_number() over(partition by DEPARTMENT_ID order by HIRE_DATE) joining_order from hr.employees H;`

②

select H.*, row_number() over(partition by DEPARTMENT_ID order by HIRE_DATE) joining_order from hr.employees H

③ ① → Dept change start again

Boundary

AME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID	JOINING_ORDER
r	Whalen	JWHALEN	515.123.4444	17-SEP-03	AD_ASST	4400	-	101	10	1
	Hartstein	MHARTSTE	515.123.5555	17-FEB-04	MK_MAN	13000	-	100	20	1
	Fay	PFAY	603.123.6666	17-AUG-05	MK_REP	6000	-	201	20	2
	Raphaely	DRAPHEAL	515.127.4561	07-DEC-02	PU_MAN	11000	-	100	30	1
er	Khoo	AKHOO	515.127.4562	18-MAY-03	PU_CLERK	3100	-	114	30	2
	Tobias	STOBIAS	515.127.4564	24-JUL-05	PU_CLERK	2800	-	114	30	3
	Baida	SBAIDA	515.127.4563	24-DEC-05	PU_CLERK	2900	-	114	30	4

← Assign data

Scenario: Finding the Top 3 Highest-Paid Employees in Each Department

Let's say we have an Employees table with the following columns:

emp_id (Primary Key)
emp_name
department_id
salary

We want to **give a numbering employees within each department based on their salary in descending order** and fetch the **top 3 highest-paid employees from each department**.

```
SELECT emp_id, emp_name, department_id, salary,
       ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY salary DESC) AS numbering
FROM Employees
```

Check the above sql. What is happening here ? The first 4 columns as it is from main table (Input preserved) and then I gave a numbering depends on salary.

ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY salary DESC) AS numbering

1. PARTITION BY department_id : It executes first and creates a boundary.
2. ORDER BY salary DESC : within this boundary, all salaries are arranged in descending order.
3. ROW_NUMBER() : it will come in last and give numbering. once department change starts again.

Now, we have all data and I want **top 3 highest-paid employees**. Means **numbering <= 3**

```
With CTE as (
  SELECT emp_id, emp_name, department_id, salary,
         ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY salary DESC) AS numbering
  FROM Employees
) select * from CTE where numbering <= 3;
```

Fetching the Latest Order of Each Customer

Let's say we have an Orders table and it contains entry for each customer. one customer can have multiple orders. with the following columns:

order_id (Primary Key)
customer_id
order_date
amount

```
SELECT order_id, customer_id, order_date, amount,
       ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DESC) AS row_num
FROM Orders;
```

```
ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DESC) AS row_num
```

1. `PARTITION BY customer_id` : It executes first and creates a boundary for each customer.
2. `ORDER BY order_date DESC` : within this boundary, all order dates are arranged in descending order.
3. `ROW_NUMBER()` : it will come in last and give numbering. once `customer_id` change starts again.

after that just choose where `row_num = 1`